Speeding Up Autoregressive LLM Generation via Speculative Decoding: Principle, Self-Implementation, and Evaluation

Zou Bowen (邹博闻) and Cui Di (崔迪)
College of Computer Science, Nankai University
Team: 创造性 AI 2312251@mail.nankai.edu.cn 19837008208@163.com

1 Background and Motivation

Autoregressive language models generate text one token at a time: each new token requires a full forward pass over the entire token history. For large models this step is *memory-bandwidth bound*—reading the weight matrices from memory each step dominates compute—which limits throughput to tens of tokens per second on a single accelerator.

Speculative decoding (Leviathan et al., 2023; Chen et al., 2024) reduces the number of expensive target-model forward passes without changing the output distribution. A cheap *draft model* proposes K speculative tokens; the target model verifies them in a *single* parallel forward pass, and tokens are accepted probabilistically so that the final distribution is identical to vanilla autoregressive sampling. Because small models agree with large ones on most frequent short tokens—a statistical property of natural-language token distributions—most drafts are accepted, yielding often 2–3× speedups *at zero quality cost*.

Rather than merely *using* an existing engine (explicitly forbidden by the project rules), this project implements the speculative-decoding pipeline *from scratch* over an open-source LLM, rigorously verifies the "lossless" claim, and delivers a runnable interactive demo.

2 Problem Definition and Research Questions

Given a target LM p, a draft LM q, a prompt, and draft length K, we implement a speculative decoding loop whose generated distribution equals vanilla autoregressive decoding. We ask:

RQ1. Can a self-implemented engine reproduce the claimed lossless speedup (exact output preservation) in both greedy and sampling modes?

RQ2. How do drafting strategies (small LM vs. n-gram prompt lookup) and hyper-parameters (K, temperature) affect throughput, acceptance rate, and oracle-like quality?

RQ3. How do gains scale with the draft/target model pair, sequence length, and serving batch size?

3 Technical Approach and System Design

Core modules (all self-implemented).

1. **Decoder engine**: KV-cache autoregressive generation, greedy and sampling modes (the baseline).
2. **Draft module**: K-step continuous sampling from a small draft LM; plus an n-gram prompt-lookup variant that proposes tokens copied from the prompt (no second model needed) (NVIDIA, 2023).
3. **Validator**: batch forward pass of the target model over the K proposed positions; per-position acceptance with probability $\min(1, p(\cdot)/q(\cdot))$, resampling otherwise.
4. **Eval harness**: throughput, latency, acceptance rate, and distribution-preservation metrics.

Architecture and implementation. The engine is written in PyTorch and validated first on CPU (numerical correctness and unit tests), then deployed on the in-lab Ascend 950 device which we already have running an open-source inference stack; the speculative loop itself is independent of vendor kernels. We deliberately *do not* use the speculative decoding built into vLLM or similar frameworks;

the baseline is the same engine with drafting disabled. A command-line runner and a lightweight web demo expose a toggle between vanilla and speculative decoding with live tokens/s and acceptance stats.

4 Evaluation Plan

Setup. Target models: Qwen3-8B (primary) and Qwen2.5-7B (generality). Drafts: Qwen3-0.6B, Qwen2.5-0.5B, and n-gram prompt lookup. A fixed prompt set of 20–50 samples across domains (writing, code, news).

Metrics.

1. *Speed:* end-to-end speedup vs. baseline, tokens/s, time-to-first-token.
2. *Acceptance:* accepted/proposed ratio, drafts "all-correct" fraction.
3. *Quality preservation (the key claim):* greedy outputs must match vanilla *token-by-token*; sampling outputs measured by KL divergence / perplexity to the vanilla distribution.

Ablations. Draft length $K \in \{1, \ldots, 8\}$; draft strength (0.6B vs. 0.5B vs. n-gram); temperature; batch size.

5 Expected Outcomes

1. A clean, self-implemented, reproducible speculative-decoding engine with unit tests and documented code.
2. Quantitative lossless speedups with systematic ablations and acceptance-rate analysis, reported against an honest baseline.
3. A runnable interactive demo (web), the 8-page ACL-style final report, slides, demo video, and a source archive with README.

6 Project Timeline

Phase	Dates	Deliverables
Phase 1	2026/10/12	Project proposal (this document, 2pp)
Phase 2	10/12–10/31	Engine implemented; first speedup measured on CPU/NPU
Phase 3	11/01–11/20	Systematic experiments and ablations; figures
Phase 4	11/21–11/30	Interactive demo; midterm report (2–3pp) + slides
Phase 5	12/01–final	8-page ACL final report, final slides, demo video, code zip

7 Team Members and Responsibilities

Member	Role	Responsibili
Zou Bowen (2312251)	Developer (Lead)	Core implem decoding engi 2); experiment ysis
Cui Di (2412439)	Reviewer, writer & presenter	Literature co work); final 8 sentation slide

References

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2024. Accelerating large language model decoding with speculative sampling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 3123–3131.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286.

NVIDIA. 2023. Accelerating llm inference with prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding. Accessed 2026-10-12.